

Week 3

Navigation & State Management

Pemrograman Mobile

Flutter • GoRouter • Riverpod

| Dari aplikasi satu layar → aplikasi multi-halaman yang terstruktur

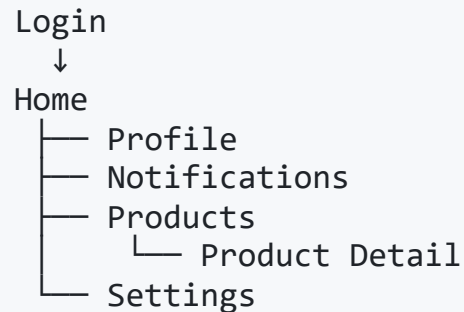
Learning Outcomes

Setelah mengikuti pertemuan ini, mahasiswa mampu:

1. Menjelaskan konsep navigation pada aplikasi mobile.
2. Mengimplementasikan navigation menggunakan `GoRouter` .
3. Membedakan local state, shared state, dan asynchronous state.
4. Mengimplementasikan state management menggunakan `Riverpod` .
5. Menangani state `loading` , `data` , dan `error` .
6. Mengintegrasikan navigation dan state management.
7. Melakukan review dan verifikasi kode yang dihasilkan AI.

Mengapa Navigation & State Management?

Aplikasi nyata hampir tidak pernah hanya memiliki satu layar.



Semakin banyak layar dan data:

- navigation menjadi lebih kompleks
- state semakin banyak
- komunikasi antar-widget semakin sulit
- kode semakin sulit dipelihara

Tujuan: membuat struktur yang tetap sederhana ketika aplikasi bertambah besar.

Mini Poll

Jika aplikasi memiliki 15 halaman...

Bagaimana Anda akan berpindah halaman?

- A. `Navigator.push()` di mana-mana
- B. Semua halaman dalam satu widget
- C. Routing yang terstruktur
- D. Meminta AI membuat seluruh navigation

Diskusi

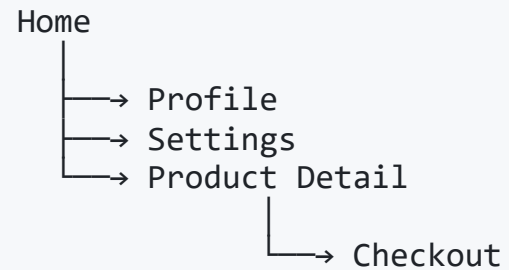
Apa masalah jika setiap halaman mengatur navigation sendiri?

Bagian 1

Navigation

Apa Itu Navigation?

Navigation adalah mekanisme untuk berpindah antar-screen/page.



Navigation harus mampu menangani:

- perpindahan halaman
- parameter
- deep link
- back navigation
- authentication guard
- nested navigation

Navigation Dasar Flutter

Flutter menyediakan `Navigator` .

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (_) => const DetailPage(),  
  ),  
);
```

Kembali:

```
Navigator.pop(context);
```

Kelebihan

- sederhana
- mudah dipahami
- cocok untuk aplikasi kecil

Kekurangan

- routing tersebar
- sulit dikelola ketika aplikasi membesar
- deep linking lebih kompleks

Mengapa GoRouter?

GoRouter

Konsep:

```
URL / Route  
  ↓  
GoRouter  
  ↓  
Page / Screen
```

Contoh:

```
/
├── /login
├── /home
├── /profile
└── /product/:id
```

GoRouter

Keuntungan:

- centralized routing
- named route
- path parameter
- query parameter
- redirect / route guard
- deep linking
- nested route

Konsep Route

Static path

```
/home  
/profile  
/settings
```

Path parameter

```
/product/123  
/product/456
```

Route:

```
/product/:id
```

Query parameter

```
/products?category=food
```

Struktur GoRouter

```
final router = GoRouter(  
  routes: [  
    GoRoute(  
      path: '/',  
      name: 'home',  
      builder: (context, state) =>  
        const HomePage(),  
    ),  
    GoRoute(  
      path: '/detail/:id',  
      name: 'detail',  
      builder: (context, state) {  
        final id =  
          state.pathParameters['id'];  
        return DetailPage(id: id!);  
      },  
    ),  
  ],  
);
```

Perhatikan: route didefinisikan di satu tempat.

Named Route

Daripada:

```
context.go('/detail/123');
```

dapat menggunakan:

```
context.goNamed(  
  'detail',  
  pathParameters: {  
    'id': '123',  
  },  
);
```

Mengapa berguna?

Jika struktur URL berubah, penggunaan named route membuat kode lebih mudah dikelola.

go() vs push()

context.go()

```
context.go('/home');
```

Mengubah lokasi saat ini.

context.push()

```
context.push('/detail/123');
```

Menambahkan halaman ke navigation stack.

```
go()  
Home —————→ Detail  
  
push()  
Home → Detail → Back → Home
```

Gunakan sesuai kebutuhan navigation stack aplikasi.

Route Parameter

Misalnya:

```
/product/101
```

Route:

```
/product/:id
```

Ambil parameter:

```
final id =  
    state.pathParameters['id'];
```

Kemudian:

```
ProductDetailPage(  
    productId: id!,  
)
```

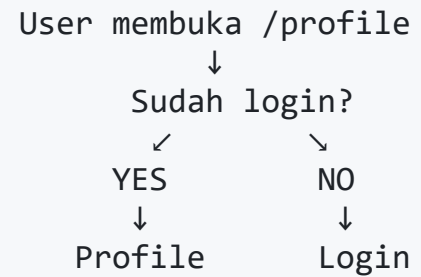
Pertanyaan

Apa yang terjadi jika `id` tidak valid?

Navigation juga membutuhkan validasi.

Route Guard / Redirect

Contoh:



Konsep:

```
redirect: (context, state) {  
  final loggedIn = authState.loggedIn;  
  
  if (!loggedIn) {  
    return '/login';  
  }  
  
  return null;  
}
```

Bagian 2

State Management

Apa Itu State?

State adalah data/kondisi yang dapat berubah selama aplikasi berjalan.

```
counter = 0  
counter = 1  
counter = 2
```

Contoh lain:

- user sedang login
- daftar produk
- tema dark/light
- item keranjang
- status loading
- hasil API
- error
- notification count

Masalah Tanpa State Management

Bayangkan:

```
HomePage
├── Widget A
├── Widget B
├── Widget C
└── Widget D
```

Semua membutuhkan:

```
user
cart
notifications
products
```

Masalah Tanpa State Management

Jika data terus dilempar:

A → B → C → D → E

Kode dapat menjadi:

- sulit dibaca
- sulit diuji
- sulit dipelihara
- coupling tinggi

Jenis State

1. Local State

Digunakan satu widget.

```
isPasswordVisible  
selectedTab  
counter
```

2. Shared / App State

Digunakan beberapa bagian aplikasi.

```
currentUser  
shoppingCart  
theme
```

Jenis State

3. Async State

Data yang prosesnya asynchronous.

Loading



Data

atau

Loading



Error

Kenapa Riverpod?

Riverpod

State management dan dependency management untuk Flutter/Dart.

```
Provider  
  ↓  
State / Dependency  
  ↓  
Widget
```

Keuntungan:

- dependency lebih eksplisit
- dapat digunakan lintas widget
- mendukung async state
- testable
- cocok untuk aplikasi yang berkembang

Provider Sederhana

```
final counterProvider =  
    StateProvider<int>((ref) => 0);
```

Membaca:

```
final count =  
    ref.watch(counterProvider);
```

Mengubah:

```
ref.read(  
    counterProvider.notifier,  
).state++;
```

ConsumerWidget

```
class HomePage extends ConsumerWidget {  
  
  const HomePage({super.key});  
  
  @override  
  Widget build(  
    BuildContext context,  
    WidgetRef ref,  
  ) {  
    final count =  
      ref.watch(counterProvider);  
  
    return Text('$count');  
  }  
}
```

Inti konsep

```
ref.watch()  
↓  
Widget bergantung pada state  
↓  
State berubah  
↓  
Widget rebuild
```

watch() vs read()

watch()

Mendengarkan perubahan.

```
ref.watch(counterProvider);
```

read()

Mengakses tanpa listen.

```
ref.read(counterProvider);
```

Sering digunakan ketika melakukan action:

```
ref  
  .read(counterProvider.notifier)  
  .state++;
```

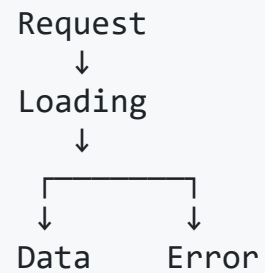
Ingat

watch = react terhadap perubahan

read = akses tanpa subscription

Async State

Data dari API tidak langsung tersedia.



Riverpod menyediakan pola:

`AsyncValue<T>`

UI dapat menangani:

Loading → Progress indicator
Data → Tampilkan data
Error → Tampilkan pesan

AsyncValue

Pola umum:

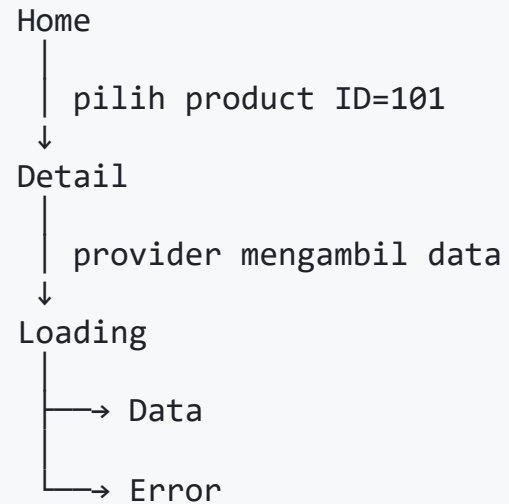
```
final state = ref.watch(  
  productsProvider,  
);  
  
return state.when(  
  loading: () =>  
    const CircularProgressIndicator(),  
  
  data: (products) =>  
    ProductList(products),  
  
  error: (error, stack) =>  
    Text('Error: $error'),  
);
```

Keuntungan

Kondisi asynchronous menjadi eksplisit.

Navigation + State

Gabungkan keduanya:



Navigation menentukan:

halaman mana yang ditampilkan

State management menentukan:

data dan kondisi apa yang ditampilkan

Studi Kasus

ToDo Application

```
Home
├── Add Task
├── Task Detail
└── Filter
```

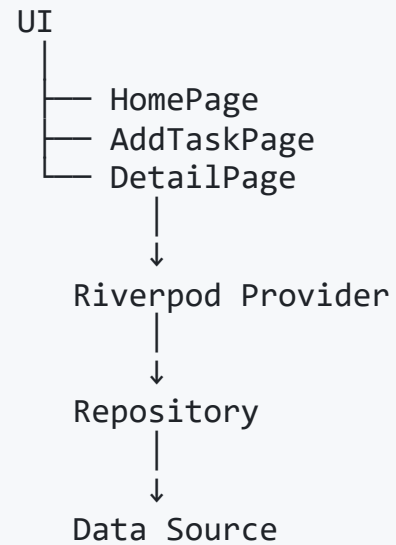
State:

```
tasks
filter
loading
error
```

Contoh:

```
All
├── Belajar Flutter ✓
├── Mengerjakan tugas
└── Review kode AI
```

Arsitektur Sederhana



Untuk Week 3, fokus utama:

Navigation + State Management

Clean Architecture dibahas lebih mendalam pada Week 7.

Anti-Pattern

Hindari:

1. Semua state di satu widget

```
HomePage  
└─ 500 lines of code
```

2. Semua data dilempar melalui constructor

```
A → B → C → D → E
```

3. State global untuk semua hal

Tidak semua state harus global.

4. AI-generated code tanpa verifikasi

Kode terlihat benar ≠ kode benar.

AI Lab

Gunakan AI sebagai Co-Developer

Contoh prompt:

Saya membuat aplikasi Flutter.

Gunakan:

- GoRouter
- Riverpod
- ConsumerWidget

Buat navigation:

Home → Detail/:id

State detail harus memiliki:

Loading, Data, Error.

Jelaskan alasan setiap bagian kode.

Jangan berhenti pada output AI.

AI Verification Challenge

Setelah AI menghasilkan kode:

1. **Baca** — pahami setiap bagian.
2. **Verifikasi** — cek API dan pola yang digunakan.
3. **Jalankan** — pastikan aplikasi benar-benar berjalan.
4. **Uji** — coba kondisi normal dan error.
5. **Refactor** — sederhanakan jika diperlukan.
6. **Jelaskan** — mampu menerangkan kode tanpa AI.

Uji minimal

- ID kosong
- ID tidak ditemukan
- API error
- user menekan Back
- state berubah

Aktivitas Kelompok

Code Review — 15 Menit

Cari minimal:

- 2 potensi bug
- 2 masalah maintainability
- 1 masalah state
- 1 masalah navigation
- 1 improvement dari kode AI

Kemudian jawab:

Apakah kode ini layak masuk production? Mengapa?

Quiz Cepat

1. Apa fungsi GoRouter?

- A. Database
- B. Navigation/routing
- C. HTTP client
- D. Package manager

2. `ref.watch()` digunakan untuk?

- A. Menghapus provider
- B. Mendengarkan perubahan state
- C. Menutup aplikasi
- D. Membuat route

3. `AsyncValue` cocok untuk?

- A. Data asynchronous
- B. Warna aplikasi
- C. Icon
- D. Padding

Jawaban Quiz

1 → B

GoRouter digunakan untuk routing/navigation.

2 → B

`watch()` membuat widget bereaksi terhadap perubahan provider.

3 → A

`AsyncValue` cocok untuk merepresentasikan:

Loading
Data
Error

AI Challenge

Gunakan AI untuk membuat **versi awal**:

“Buatkan struktur GoRouter dan Riverpod untuk aplikasi ToDo Flutter dengan Home, Add Task, Detail Task, dan Settings.”

Kemudian mahasiswa wajib:

1. memahami kode
2. menjalankan kode
3. menemukan minimal 2 kelemahan
4. memperbaiki kode
5. menambahkan fitur sendiri
6. mencatat perubahan manual

Deliverables

Repository

```
week-03-navigation-state/  
├── lib/  
├── test/  
├── README.md  
└── screenshots/
```

README minimal berisi:

- deskripsi aplikasi
- route/provider yang digunakan
- screenshot
- cara menjalankan
- AI tools yang digunakan
- prompt utama
- perubahan manual
- masalah yang ditemukan dan hasil testing

Rubrik Week 3

Komponen	Bobot
Navigation / GoRouter	20%
Riverpod / State Management	25%
Async & Error State	15%
UI & UX	10%
Testing	10%
Code Quality	10%
Responsible AI Usage	10%
Total	100%

Checklist Sebelum Submit

Navigation

- ☐ Semua route berjalan
- ☐ Named route digunakan
- ☐ Parameter dapat diteruskan
- ☐ Back navigation berjalan
- ☐ Invalid route ditangani

State

- ☐ State ditempatkan dengan tepat
- ☐ Provider digunakan dengan benar
- ☐ `watch()` dan `read()` digunakan sesuai kebutuhan
- ☐ Loading/Data/Error ditangani

AI

- ☐ Prompt dicatat & Output AI diverifikasi
- ☐ Ada perubahan manual
- ☐ Kode dipahami mahasiswa

Exit Ticket

Jawab sebelum kelas berakhir:

1. Apa perbedaan:

Navigator
vs
GoRouter

2. Kapan menggunakan:

Local State
vs
Shared State

3. Apa fungsi:

ref.watch()
ref.read()

4. Mengapa kode AI tetap harus diverifikasi?

Key Takeaways

Navigation

GoRouter membantu membuat routing lebih terstruktur.

State Management

Riverpod membantu mengelola state dan dependency secara eksplisit.

Async

`AsyncValue` membuat Loading/Data/Error lebih jelas.

AI

AI menghasilkan kode.

Developer bertanggung jawab atas kebenaran kode.

Next Week

Week 4: Networking & REST API

```
Flutter
↓
Dio
↓
REST API
↓
JSON
↓
Model
↓
Repository
↓
UI
```

Target

Aplikasi Flutter tidak hanya menyimpan data lokal, tetapi mulai berkomunikasi dengan backend.

Build → Verify → Explain

Prinsip Pembelajaran Mobile



Jangan hanya bisa membuat aplikasi.

Jadilah developer yang memahami aplikasi yang dibuat.